

TAYLOR'S UNIVERSITY

Wisdom · Integrity · Excellence

SCHOOL OF COMPUTER SCIENCE

ITS63304 & ITS69704

Object Oriented Programming

Semester-1 September 2026

Group Assignment (30%)

MLO2: Build capability to develop applications to solve computational problems related to object-oriented programming language.

Development of Java Application related to SDG

Project Title: ***Water Usage Tracker***

SDG 6 – Clean Water and Sanitation

Assignment Instructions

1. Group Formation

- This is a group assignment, and each group must consist of four (4) members.
- All members are expected to contribute equally, work collaboratively, and actively participate in completing the assignment.
- Students are encouraged to share ideas, engage in discussions, and brainstorm collectively to achieve the best outcomes.
- In cases where a group member is found to be inactive or not contributing fairly, marks may be deducted. This will be determined based on the situation and the declarations provided in the Individual Tasks Allocation Form.

2. Ethics and Academic Integrity

- All submissions must be original work.
- Proper citation and references must be provided for any material sourced from publicly available resources.
- The Academic Honors Code strictly applies. Any form of plagiarism or collusion will result in severe penalties, including zero marks.

3. Submission Guidelines

- Each group must submit one report only, in PDF format, and project file (codes) via MyTimes by end of Week 12.
- Any approved extensions will be formally communicated through MyTimes.
- The report must include:
    - Full names and student identification numbers of all group members
    - Details clearly stated on both the cover page and the first page
    - The official assignment cover page template provided on MyTimes must be used.
    - The declaration on the cover sheet must be signed by all group members.

4. Documentation and Submission Requirements

4.1 Report Preparation

1. Use the provided assignment cover page template, including complete details of all group members.
2. Ensure all Java code is well-documented, with detailed comments explaining:
    - Functionality and logic
    - Expected outputs
    - Key considerations or potential issues
3. All work must be original and properly referenced using the APA Referencing System.
4. All external information, figures, and diagrams must be appropriately cited.

4.2 Plagiarism Check

1. Prior to final submission, upload your report to Turnitin.
2. The similarity index must be below 20%.
3. Attach the one-page Turnitin similarity report to your assignment.

4.3 Required Documents

1. Submit the Assessment Submission Declaration Form, for each group member.
2. Submit "Individual Tasks Allocation Form", signed by all members.
3. Combine all declaration forms and the assignment report into a single ZIP file.
4. Submission must be made via MyTimes only.

4.4 General Requirements

1. Ensure the assignment is submitted by the deadline specified by the instructor.
2. Conduct a final review to confirm completeness and correctness of the document.
3. Note: Submission does not guarantee a passing grade.

5. Group Management Rules

- Each group must:
  - Have a group name (e.g., Group 1), approved by the instructor
  - Appoint a team leader
  - Consist of members from the same practical group/section
- All members must sign the declaration form and **Individual Tasks Allocation Form**.
- Group changes are not permitted once finalized.
- Any conflicts must be resolved internally within the group.
- Groups must be formed promptly when instructed. Failure to form a group may result in no marks being awarded.

6. Special Cases

- If a group is reduced to one member (due to withdrawal/transfer), the remaining student must complete all assignment requirements independently.

7. Incomplete Submission

Examples include:

- Missing report or source code
- Missing Turnitin report
- Incomplete cover page details

Penalties:

- Failure to submit report or source code: Zero marks
- Other incomplete submissions: Marks capped at B grade

8. Failure to Comply

Examples include:

- Exceeding similarity threshold
- Improper APA referencing

Penalty: Marks capped at B- grade

9. Late Submission Policy

- The late penalty will take the form of a deduction of a fixed percentage of marks **(5%)** per day up until a maximum period of five (5) days, at which time the assignment will be assigned zero marks.
- For late submission of assignments, Saturday, Sunday and public holidays may be counted identically as any other working day.

10. Guidelines for Allocation of Assignment Marks in Group Work

- The evaluation of assignment marks will be based on both group efforts and individual contributions. The distribution of marks is outlined below:

A. Group Effort Components

- The following sections will be assessed as group work, and all members of the group will receive the same marks for these components:
  1. Introduction
  2. Problem Statement
  3. Discussion and Reflection
  4. Conclusion
  5. References

B. Individual Effort Components

- The following sections will be evaluated as individual contributions, and marks will vary from one student to another based on the quality, originality, and depth of each student's work:
  1. Application Design
  2. Implementation
  3. Output and Testing
  4. Presentation

C. General Note

- Group effort components reflect the team's collective understanding, coordination, and collaboration.
- Individual effort components assess each student’s personal contribution, technical competence, and level of engagement with the assignment.

Students are therefore advised to actively participate in both group discussions and individual tasks to achieve balanced performance across all evaluation criteria.

Water Usage Tracker

SDG 6: Clean Water and Sanitation

1. Project Introduction

Access to clean water and its sustainable management is the key objective of Sustainable Development Goal 6 (SDG 6). Wasteful household water use increases utility bills, puts pressure on limited freshwater resources, and contributes to water scarcity.

In this project, you will design and develop a simple console-based Java application that helps households monitor water consumption, identify high-usage activities and fixtures, and promote water-saving behavior.

The project emphasizes object-oriented programming (OOP) concepts while solving a real-world sustainability problem.

2. Learning Objectives

By completing this project, you will be able to:

1. Define and use classes and objects to model household water-related entities.
2. Apply encapsulation using private attributes and public getter/setter methods.
3. Implement inheritance to model different types of water-consuming fixtures or activities.
4. Demonstrate polymorphism through method overriding and dynamic method calls.
5. Write simple, readable, and maintainable Java code to address a real-world SDG problem.

3. Problem Statement

Many households use far more water than they realise, because daily activities such as showering, flushing toilets, doing laundry, and watering the garden are rarely measured. This leads to higher utility bills, wasted freshwater, and added pressure on limited water resources. The task is to design a Java application that allows households to record their daily or monthly water consumption and track the usage of different fixtures and activities, such as the shower, toilet, washing machine, and garden hose. The application should calculate total water usage and estimated cost, and it should help users encourage reduced consumption and water-saving habits by identifying which fixtures use the most water.

4. Functional Requirements

The application must allow users to add and store the water-using fixtures in their household, such as showers, toilets, washing machines, and garden hoses, and to track the water each one consumes in litres. It should display summary reports showing the total water used, the average usage per day, and the estimated bill, so that households can easily see their consumption patterns. The application must also identify high-water-consuming fixtures, so that users know where they can save the most water. To keep the design organised and easy to extend, it must support multiple fixture types using object-oriented programming principles such as inheritance and polymorphism.

5. Project Design Requirements

5.1 Class Design (Classes and Objects)

You must design at least 4 to 6 Java classes. Suggested classes:

Class	Role
WaterFixture	Base (super) class
ShowerFixture	Subclass: usage based on flow rate and minutes
ToiletFixture	Subclass: usage based on litres per flush and number of flushes
ApplianceFixture	Subclass (e.g., washing machine): usage based on litres per cycle and number of cycles
WaterUsageRecord	Stores a dated usage entry
Household	Holds the list of fixtures and records
WaterTrackerApp	Main class with menu

Each class should:

- Represent a meaningful real-world entity
- Contain attributes and methods relevant to water consumption
- Along with the basic requirements, also to include association relationships in the example

5.2 Encapsulation

All data members must be:

- Declared as private
- Accessed and modified only through public getter and setter methods

Example attributes:

- Fixture name
- Flow rate (litres per minute) or volume per use (litres)
- Usage duration (minutes) or number of uses
- Water consumed (litres)

Include basic validation, for example:

- Water usage cannot be negative
- Fixture name cannot be empty
- Number of uses cannot be zero or negative

5.3 Inheritance

You must use inheritance to reduce code duplication and model specialization.

Required structure:

- One base (super) class
- At least two derived (sub) classes

Each subclass should:

- Inherit common attributes (e.g., name, location in house)

- Add or specialize behavior where appropriate (e.g., ShowerFixture adds flow rate and minutes)

5.4 Polymorphism

You must demonstrate polymorphism by:

- Defining a method in the base class (e.g., calculateWaterUsage())
- Overriding it in the subclasses
- Using parent-class references to call subclass methods at runtime

Example idea:

- ShowerFixture: flow rate × minutes
- ToiletFixture: litres per flush × number of flushes
- ApplianceFixture: litres per cycle × cycles

```
WaterFixture f = new ShowerFixture("Master Bath Shower", 9.5, 8);
System.out.println(f.calculateWaterUsage()); // calls the overridden version
```

5.5 Application Logic and Problem Solving

Your program should include:

- Arithmetic calculations (e.g., total litres, cost = litres ÷ 1000 × tariff per cubic metre)
- Conditional statements (e.g., flagging a fixture as "high consumption" if it exceeds a set threshold)
- Looping constructs (for menus, lists, summaries)

The logic should be:

- Simple
- Clearly structured
- Easy to follow

5.6 User Interaction

The program should allow user interaction through a console-based menu using Scanner. Menu options such as:

1. Add fixture
2. Record water usage
3. View total consumption and estimated cost
4. Show high-consumption fixtures and saving tips
5. Exit program

5.7 Main Program Execution

Your main() method must:

- Create objects from different classes

Demonstrate:

  - Object creation
  - Getter and setter usage
  - Inheritance
  - Polymorphic behavior
- Display clear output showing total water usage and insights

Sample output:

```
===== Monthly Water Report =====
Shower            : 2,280.0 L
Toilet            : 1,620.0 L
Washing Machine   :   960.0 L
-------------------------------
Total             : 4,860.0 L
Daily Average     :   162.0 L
Estimated Cost    : RM 9.72
Highest User      : Shower (46.9%)
Tip: Reducing shower time by 2 minutes could save about 570 L per month.
```

5.8 Code Quality Requirements

Your code must:

- Follow Java naming conventions
- Be properly indented and commented
- Avoid redundant and duplicated code
- Use meaningful variable and method names

6. Assessment Focus

Aspect	Description
Classes & Objects	Proper use of class structure and object instantiation
Encapsulation	Data members are private with appropriate getters/setters and validation
Inheritance	Correct use of superclass and subclass relationships
Polymorphism	Effective demonstration of method overriding and parent-class references
Application Logic	Correct calculations, conditionals, and loops; working menu
SDG Link	Application purpose clearly aligns with SDG 6 (Clean Water and Sanitation)

Appendix A



ITS63304 **Object Oriented Programming**

Semester-September 2026

Group Project (30%)

I declare that:

- I understand what is meant by plagiarism.
- The implication of plagiarism has been explained to us by our lecturer.
- This project is our own work, and we have acknowledged any use of the published or unpublished works of other people accordingly.

Name	Student ID	Signature	Marks (For Lecturer Use)
1.			
2.			
3.			
4.			

Appendix B

ITS63304 **Object Oriented Programming**

Individual Tasks Allocation

Indicate (✓) in the member name column if he/she has been involved in that task.
Add rows if necessary.

	Tasks	**Name 1 & Signature**	**Name 2 & Signature**	**Name 3 & Signature**	**Name 4 & Signature**
1.					
2.					
3.					
4.					

Appendix C
REPORT WRITING TEMPLATE

Title Page

- ✓ **Project Title:** Water Usage Tracker
- ✓ **Course Name:** Object-Oriented Programming
- ✓ Student Names and Student IDs
- ✓ Program, Semester & Instructor
- ✓ Submission Date

1. **Introduction (10 marks)**
    1. Briefly describe the **background and motivation** for the project.
    2. Explain the relevance of the project to **SDG 6: Clean Water and Sanitation**
    3. State the **main objectives** of your application.

2. **Problem Description (5 marks)**
    1. Define the problem statement in your own words.
    2. Discuss why problem is important and how program provides a computational solution.
    3. Identify the users or target audience of your application.

3. **Application Design (20 marks)**

Describe how **OOP concepts** were used in your design: **Classes and Objects.**

1. Encapsulation – how data protection was ensured.
2. Inheritance – what parent–child relationships were established.
3. Polymorphism – how dynamic behavior was achieved.
4. Include: A brief explanation of how the classes interact.

4. **Implementation (20 marks)**
    1. Describe the programming language and environment used (e.g., Java, Eclipse/VS Code).
    2. Provide key code snippets showing OOP features implementation (not the full code).
    3. Explain how the application executes and handles basic user input/output.

5. **Output and Testing (15 marks)**
    1. Show sample output (screenshots or console results).
    2. Explain how you tested your application (e.g., sample inputs).
    3. Mention any issues or limitations encountered.

6. **Discussion and Reflection (10 marks)**
    1. Reflect on how OOP principles helped solve the problem.
    2. Describe what you learned from the project.
    3. Suggest possible extensions or improvements for the future.

7. **Conclusion (10 marks)**
    1. Summarize what was achieved.
    2. Reaffirm how the application contributes to SDG 6.

8. **References (10 marks)**
    1. List all sources used (books, tutorials, websites).
    2. Use a consistent citation format (APA).

Group Assessment Evaluation Rubric (Total: 100 Marks)

Criteria (Weight)	Description	5 – Excellent	4 – Good	3 – Satisfactory	2 – Needs Improvement	1 – Poor
1. Introduction & SDG Link (10)	Clarity of project background, relevance to SDG 6, and stated objectives	Clear, insightful background. Strong, specific link to SDG 6 (e.g., water scarcity, wastage). Objectives are measurable	Clear background and objectives. SDG 6 link is valid but lacks depth	Background present but general. SDG 6 link is stated without explanation	Unclear background. Weak or vague SDG link	Missing or unrelated to SDG 6
2. Problem Description (5)	Identification of the problem, scope, and target users	Realistic, well-explained problem. Scope and households as users clearly defined	Problem well explained, but scope or users lack detail	Basic problem stated. Scope or users only partly addressed	Problem vague or unrealistic. Little on scope or users	Minimal or absent
3. OOP Design (20)	Application of classes, encapsulation, inheritance, and polymorphism	Other advanced design to include associations along with all four principles correctly applied and clearly explained (e.g., WaterFixture base class, overridden calculateWaterUsage(), validated private fields)	Most principles applied correctly. Minor gaps in explanation	Some principles applied. One is missing or misused	Only basic classes. Inheritance or polymorphism incorrect or superficial	Little or no OOP structure
4. Implementation Quality (20)	Code structure, readability, and correctness	Fully functional, modular and well-commented. Follows naming conventions. No duplicated code	Functional with minor issues in style, comments, or duplication	Basic functionality works. Some structure or readability problems	Partly functional. Frequent errors or poorly organised code	Non-functional or unclear code
5. Output & Testing (15)	Correctness and clarity of program output, and testing evidence	Accurate, validated output (totals, averages, cost, high-usage flags). Multiple test cases, including invalid input	Accurate output with clear display. Some testing shown	Output mostly correct. Basic testing only	Output has errors or is unclear. Testing minimal	No output or no testing
6. Discussion & Reflection (10)	Understanding of OOP concepts and	Insightful reflection on how OOP helped solve the	Good reflection with some specific examples	General reflection with few examples	Brief or superficial summary	Missing or vague

Criteria (Weight)	Description	5 – Excellent	4 – Good	3 – Satisfactory	2 – Needs Improvement	1 – Poor
	personal learning reflection	problem. Challenges and lessons clearly discussed				
7. Conclusion (10)	Summary of outcomes and link back to SDG 6	Concise summary. Clearly shows how the tool promotes water conservation. Future improvements suggested	Clear summary with a reasonable SDG link	Summary present but weak SDG link	Incomplete or unfocused conclusion	Missing
8. Presentation & References (10)	Clarity, formatting, organisation, and referencing style	Professional, well-organised and well-formatted. References consistent and correct	Minor formatting or referencing issues	Some formatting errors. Referencing inconsistent	Poorly organised. Few or incorrect references	Unstructured, or no references